

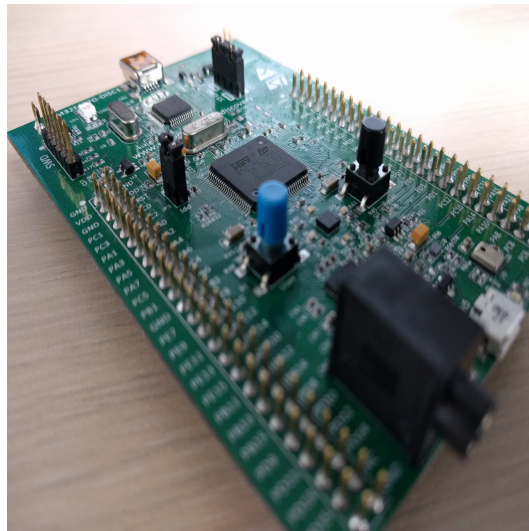
Systèmes Embarqués (SM57)

...

Sujets de Travaux Pratiques

Alexandre RAVEY

6 mars 2023



`alexandre.ravey@utbm.fr`



utbm
université de technologie
Belfort-Montbéliard

TP N°1- Prise en main IDE SW4STM32

1 Introduction

Ce TP a pour but de vous introduire les outils utilisés dans l'UV SM57 : Systèmes Embarqués. Pour ce faire, nous allons utiliser l'IDE STM32CubeIDE. Le logiciel SystemWorkbench peut également être utilisé pour réaliser les TP de SM57. Une vidéo explicative du fonctionnement des deux logiciels est disponible sur Youtube : <https://www.youtube.com/watch?v=GIBx43cJEde>

2 Démarrage du logiciel, création du Workspace

Pour démarrer l'IDE, double cliquez sur l'icône du logiciel correspondant :



Figure 1 – Icône de l'IDE sur le bureau

Au démarrage, le logiciel vous demande de lui spécifier un répertoire de travail : Si vous travaillez sur un pc UTBM, profitez en pour créer votre propre répertoire (disque de donnée / SM57 / Votre nom) qui vous servira tout au long du semestre. Vous pourrez pour chaque TP créer un projet dans ce répertoire. **Ne jamais cocher la case : "Use this as the default and do not ask again"**

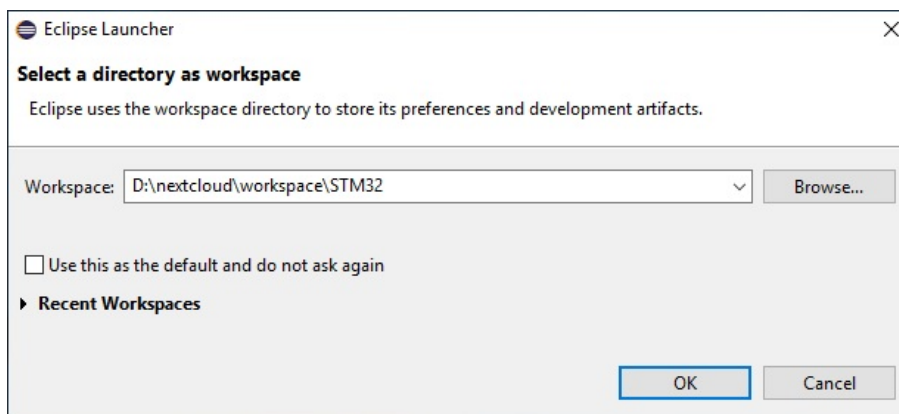


Figure 2 – Choix du répertoire de travail

Attention : Bien différencier le workspace, qui est le répertoire dans lequel nous allons déposer tous nos projet et le projet, qui est également un répertoire mais qui lui contient les fichiers sources et librairie. **Vous ne devez jamais définir un projet comme workspace, sous peine de corrompre les fichiers et de devoir tout recommencer.**

Vous devriez ensuite obtenir l'écran de bienvenue (Figure 3). Il vous suffira alors de cliquer sur *Start new STM32 project* pour continuer le TP. Remarque : Vous pouvez retrouver cet écran à tout moment en cliquant sur *Help -> Information Center*.

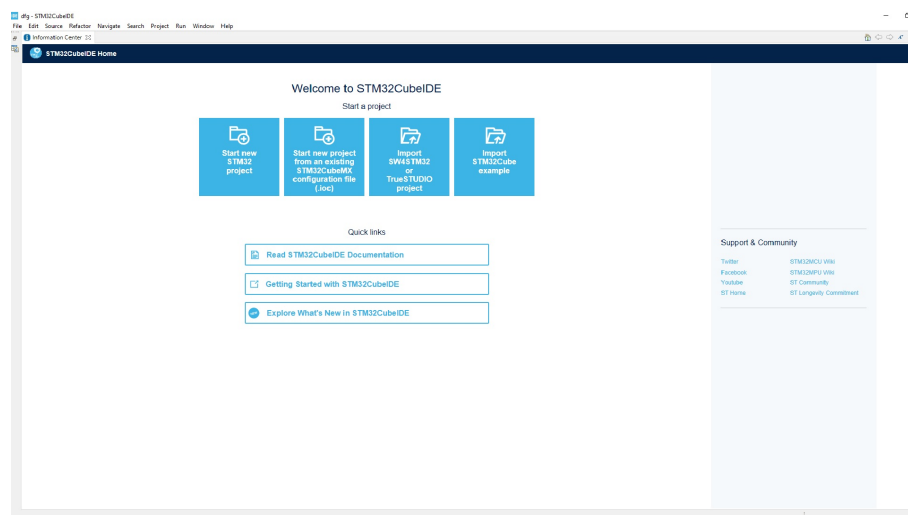


Figure 3 – L’interface vide au démarrage du logiciel

3 Introduction à la génération automatique de code C

STM32CubeMX (et son intégration dans STM32CubeIDE) est un logiciel permettant de créer automatiquement un projet pour STM32CubeIDE. De plus, le logiciel permet de générer des fonctions C pour initialiser certaines parties du micro-contrôleur. Pour ce faire, il utilise une interface graphique. L'utilisateur n'a donc pas à connaître les registres d'initialisation des divers périphériques pour utiliser le logiciel. Comme vous démarrez dans avec l'architecture STM32, vous n'avez pas encore toutes les connaissances nécessaires pour configurer les 4 LEDs de la carte ainsi que le bouton poussoir. Nous allons donc dans ce TP les configurer de façon graphique à l'aide de STM32CubeMX afin d'avoir un projet fonctionnel rapidement.

4 Création de projet sous STM32CubeMX

Le logiciel est commun à tous les micro-contrôleurs 32 bits de ST. Il est également capable de reconnaître les cartes de développements de la marque et donc la notre (STM32F407G-DISC1). Pour commencer un nouveau projet avec la configuration de la carte de TP, cliquez sur : *Access to board Selector* (figure 4)

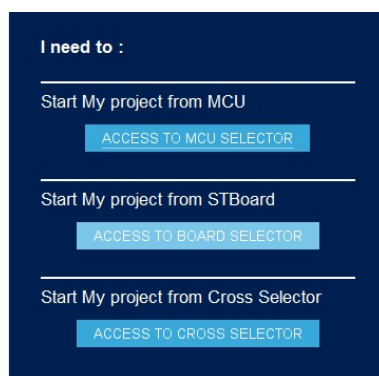


Figure 4 – Choix d'un nouveau projet en fonction de la carte de développement

Cherchez ensuite la carte correspondant à celle de TP. Attention : suivant votre version du logiciel, cette carte est nommée : STM32F4DISCOVERY (figure 5 ou STM32F407G-DISC1).

TP SYSTÈMES EMBARQUÉS (SM57)

Cliquez ensuite sur *Start Project* ou *Next*. A la question : *Initialize all peripherals with their default Mode*, répondez Oui.

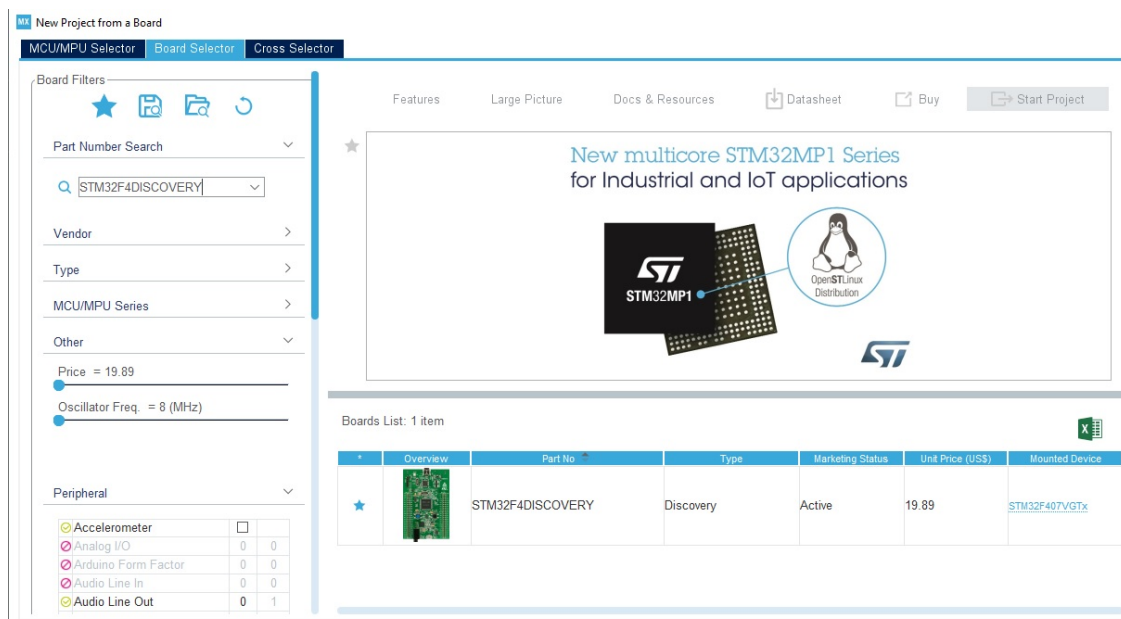


Figure 5 – Choix de la carte de TP

Vous allez voir une représentation graphique du micro-contrôleur avec toutes les entrées sorties initialisées pour réaliser les différentes fonctions que propose la carte (figure 6).

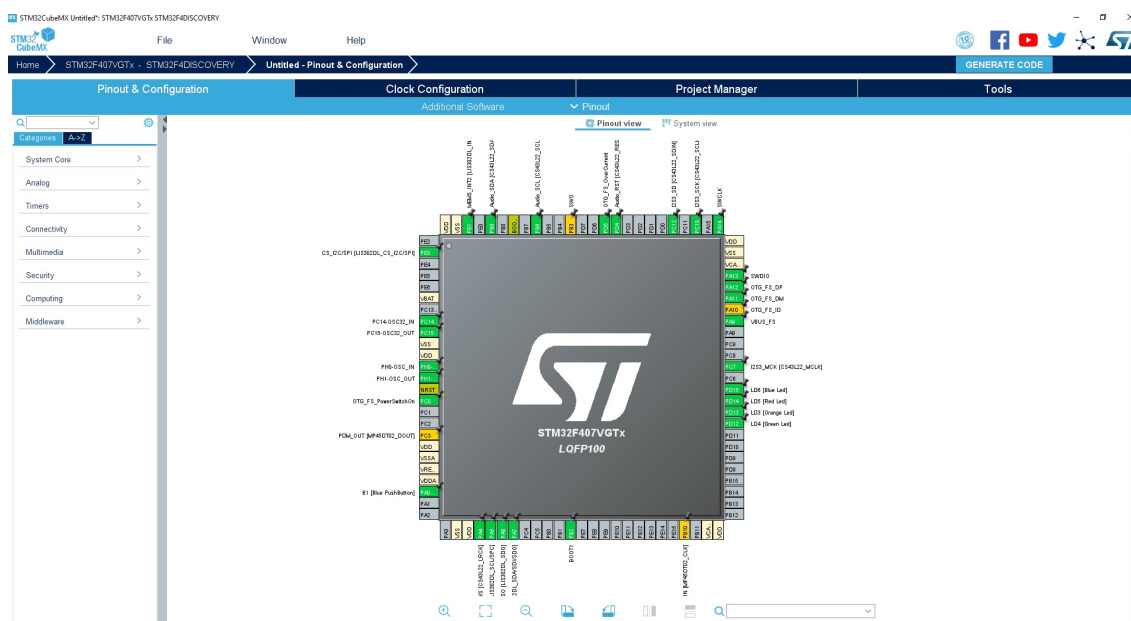


Figure 6 – Configuration par défaut

N'ayant pas besoin de toutes les fonctions, nous allons refaire nous même la configuration : Commencez par effacer la configuration actuelle en cliquant sur : *Pinout* puis *Clear Pinout*. Répondez Oui à la confirmation demandée.

Il ne reste plus qu'en couleur Jaune et Ocre les pins nécessaire à l'alimentation et au bouton Reset du micro-contrôleur.

4.1 Configuration de la fréquence d'horloge

Nous allons configurer la source de l'horloge du microprocesseur(RCC) : Afin de pouvoir fonctionner, le microprocesseur à besoin d'un signal d'horloge qu'il servira comme base, Sur la carte, nous avons plusieurs choix possible. Pour l'UV SM57, tous les projets seront basé sur le Quartz Externe de 8Mhz présent sur la carte. Cette fréquence sera ensuite modifiée pour définir la fréquence du microprocesseur (décrit par la suite). Pour configurer le quartz externe : Dans le bandeau de gauche cliquez sur *System Core* puis *RCC*. Changez ensuite le champ *High speed Clock (HSE)* par *Crystal/Ceramic Resonator* (figure 7).

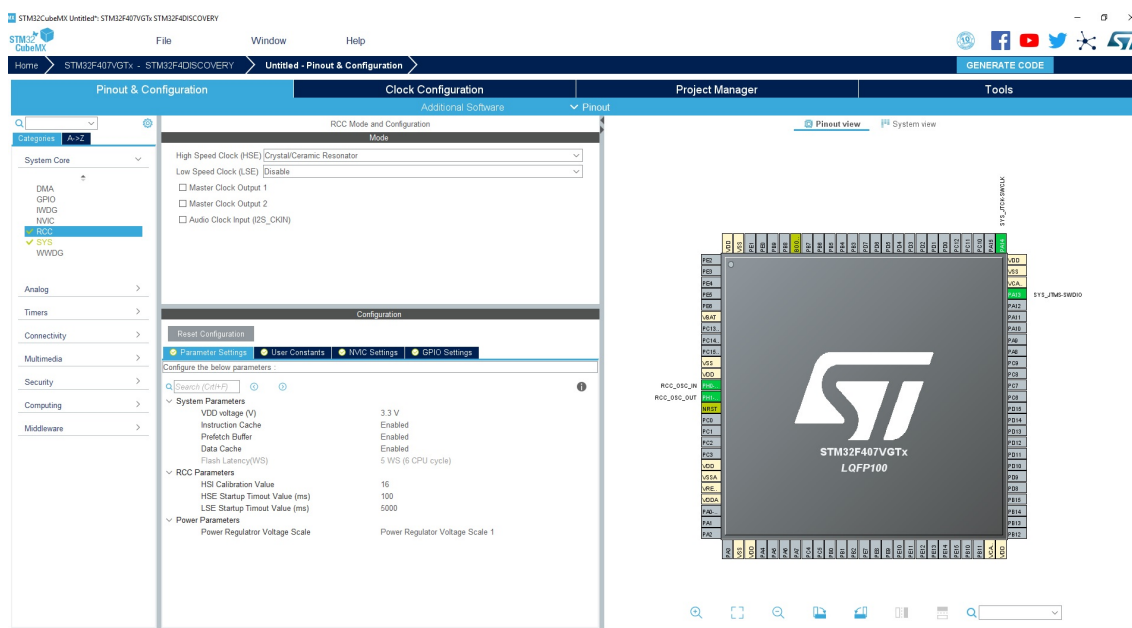


Figure 7 – Choix du quartz externe comme source d'horloge

Vous verrez des Pins attribuées à cette fonction dans la partie graphique (couleur verte). Cela sera de même pour chaque fonctionnalité paramétrée.

4.2 Configuration du lien ST-Link V2

Nous allons configurer le programmeur/débugueur : Afin de pouvoir télé-verser et déboguer le programme avec l'IDE (voir TP précédent), il est nécessaire de paramétrer le mode requis dans le micro-contrôleur. Pour ce faire, dans le bandeau de droite cliquez sur *System Core* puis *SYS*. Changez ensuite le champ Debug par Serial Wire (figure 8).

4.3 Configuration des entrées Sorties

Nous allons configurer les différentes entrées/sorties nécessaires pour ce TP (GPIO) : Afin de pouvoir utiliser les 4 LEDs ainsi que le bouton poussoir de la carte, dans un premier temps chercher dans le schéma électrique de la carte (présent sous moodle) sur quelles broches ceux ci sont reliés. Une fois identifiée, pour chaque broche, faite un clic droit dessus et choisissez : *GPIO Output* si vous voulez configurer la broche en sortie ou *GPIO Input* pour la configurer en entrée. A l'aide du bandeau de gauche naviguez ensuite dans : *System Core* puis *GPIO*. Un menu apparait avec les broches que vous venez de sélectionner. Pour chaque broche, cliquez sur sa ligne et rentrer les configurations suivantes pour les sorties(figure 9) :

- GPIO Output level : Low (La pin sera à 0V au démarrage)
- GPIO Mode : Output Push Pull (le courant est fournis par le micro-contrôleur)

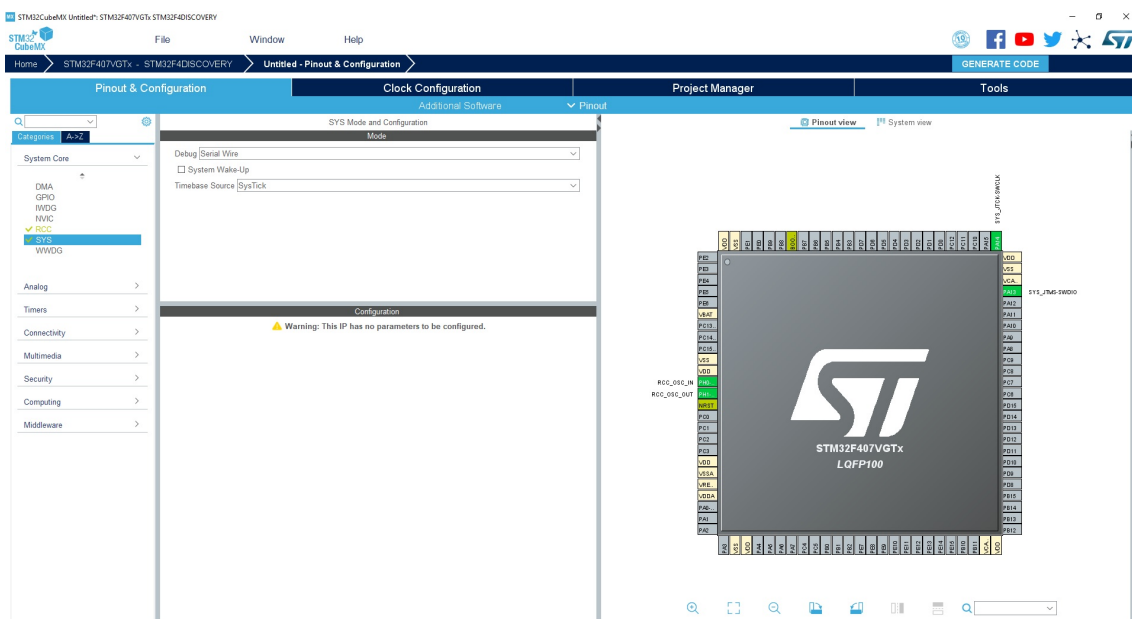


Figure 8 – Configuration du débugeur

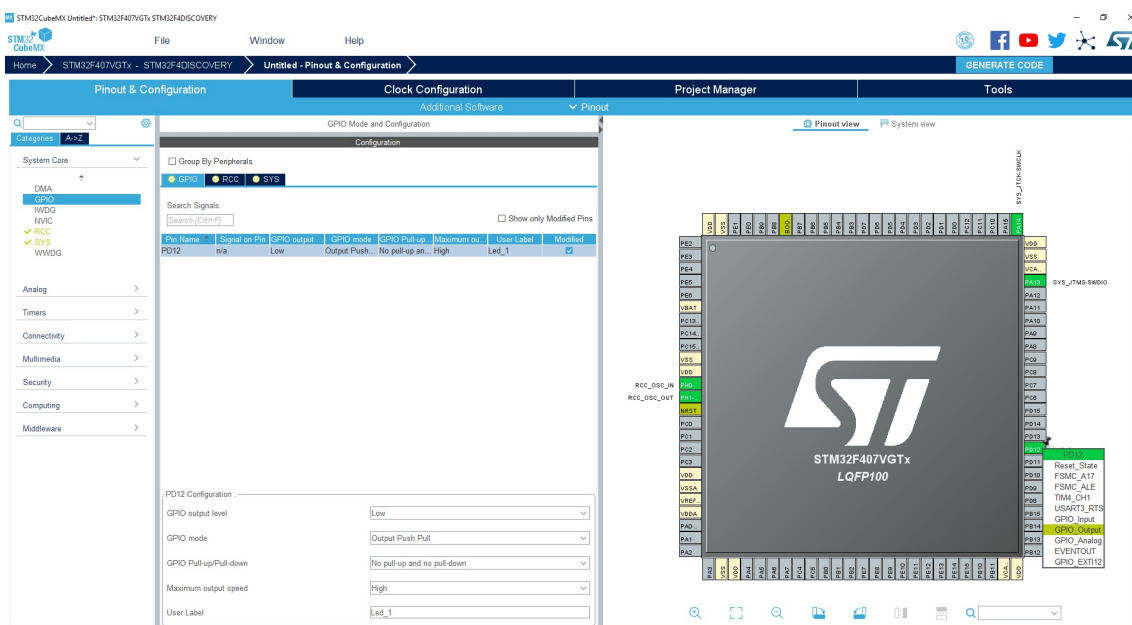


Figure 9 – Configuration d'une broche

- GPIO Pull-up/Pull-down : No PU and No PD (résistances de tirages PU/PD désactivées)
- Maximum output speed : High (le passage de 0 au 1 et inversement est fait le plus rapidement possible)
- User Label : rentrer un nom pour la branche (exemple : Led_Verte).

Pour la ou les entrées, il y a moins de paramètres, on pourra garder ceux par défaut (renseigner juste le User Label).

4.4 Réglage de la fréquence du microprocesseur

Nous allons maintenant régler les différentes horloges du micro-processeur et des périphériques du micro-contrôleur. Dans le bandeau supérieur, cliquez sur *Clock Configuration*. Vous devriez obtenir une interface correspondant à la figure 10) :

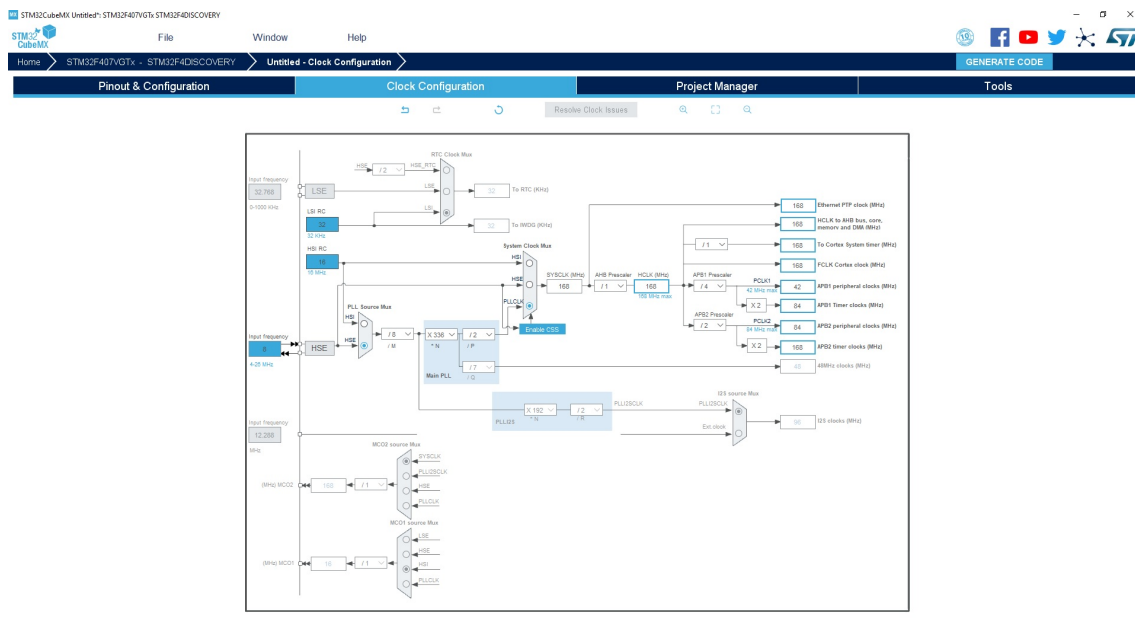


Figure 10 – Clock configuration

Analyser cette interface :

1. A quoi correspond le cadre : Main PLL ? Quel est son utilité ?
2. Dans quel rectangle se trouve l'information de la fréquence de fonctionnement du micro-processeur
3. Ou se trouve la fréquence de fonctionnement du Quartz que nous avons configuré précédemment ?

Configurez, si besoin, les paramètres de façon identique à la figure 10.

4.5 Génération du projet

Votre projet est prêt à être généré, nous allons configurer la génération automatique de code pour ce projet : Dans le bandeau supérieur, cliquez sur *Project Manager*. Différents champs doivent être renseignés : Certains champs sont déjà pré-remplis car CubeMX est lié à CubeIDE :

- *Project Name* : Renseignez le nom du projet que vous souhaitez créer.
- *Project Location* : Renseignez le votre dossier de travail (workspace)
- *Toolchain / IDE* : Utilisez le compilateur STM32CubeIDE
- *MCU Reference* : Vérifiez que la référence est bien celle de votre micro-contrôleur (STM32F407VGTx)

Garder tous les autres champs de configuration comme pré-configurés. Vous devriez obtenir une configuration proche de la Figure 11.

Cliquez ensuite sur le bouton *Generate Code* ou il suffit de faire *CTRL+S* pour enregistrer votre projet et le générer.

Une fois le projet généré, vous pouvez changer de perspective, vous devriez voir dans l'arborescence le projet crée avec les fichiers sources.

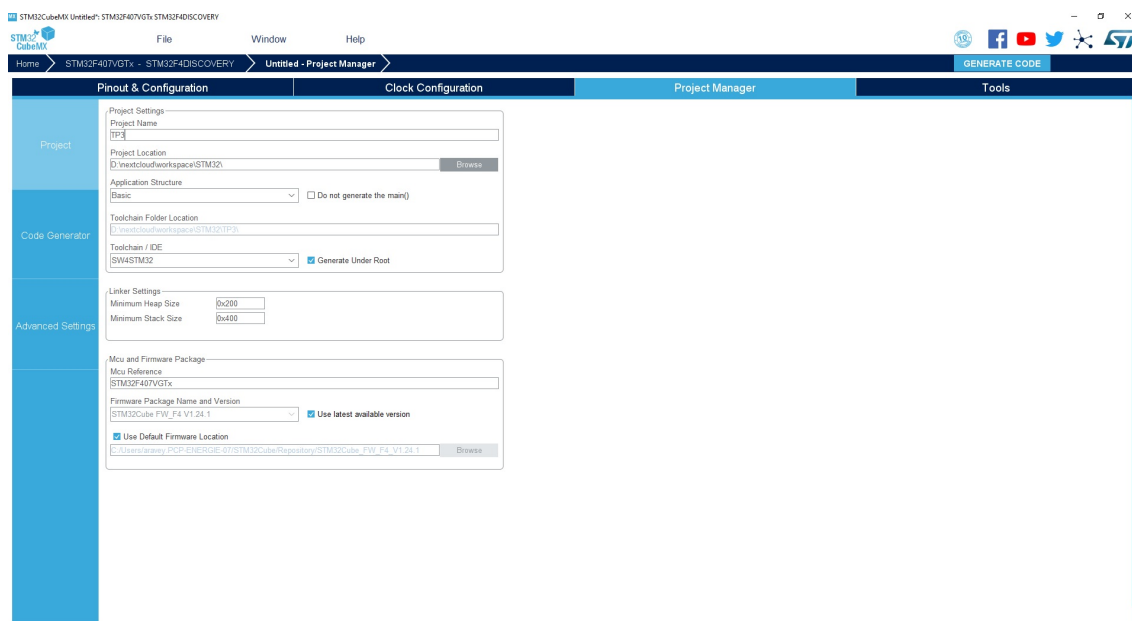


Figure 11 – Configuration de génération de projet

5 Exercice : Analyse et modification du projet créé

5.1 Découverte de l'interface de développement et des outils de debug

Dans tout projet STM32CubeIDE, il faut au minimum un fichier contenant votre programme principal. Ici, comme nous codons en C, il vous faut un fichier `.c`. Ce fichier est automatiquement créé avec la création du projet (étape précédente). Ouvrez le fichier en double cliquant sur *Src* puis double cliquez sur *main.c* Dans la partie éditeur de l'IDE (partie centrale), vous devez obtenir la même chose que la figure 12).

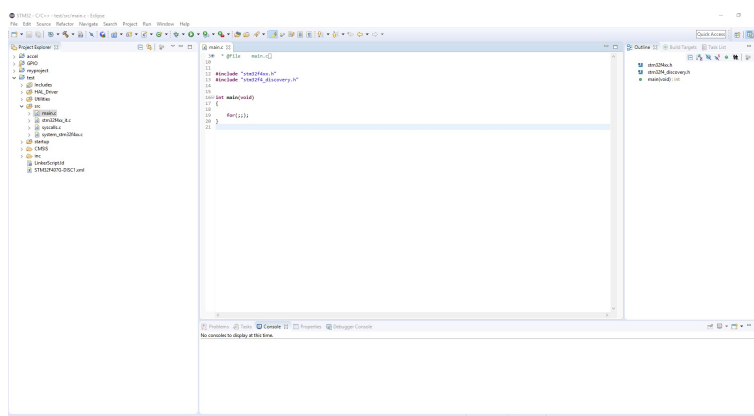


Figure 12 – Fichier main.c généré

Attardons nous un moment sur cette figure qui représente ce que vous voyez sur votre écran. La vue que vous avez du logiciel est appelée une perspective : La perspective actuelle est nommée C/C++. Cette perspective vous sert à modifier votre programme, créer et gérer des fichiers de votre projet. Elle est basée sur différentes fenêtres :

- Project Explorer : Montrant une arborescence du projet et des fichiers inclus dans celui ci
- File : La fenêtre principale (au milieu), vous permettant de modifier un fichier

- Outline / Build Targets / Task List (onglets) : Vous permettant d'avoir un résumé du code et de pouvoir en cliquant sur les éléments de directement aller à la ligne désirée
- Console / Problems / Properties / Debugger Console (onglets) : Positionnée en bas, vous permet d'avoir un retour sur le résultat de la compilation et les divers erreurs à corriger.

Il existe 3 perspectives par défaut dans STM32CubeIDE : C/C++ (celle que nous visualisons actuellement) et la perspective Debug (présentée par la suite). Vous trouverez également STM-CubeMX utilisé précédemment qui est intégré au logiciel comme une perspective. Pour basculer d'une perspective à l'autre, il suffit de cliquer sur leur icône dans la barre de tâche (voir figure 13)

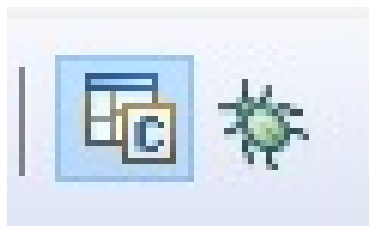


Figure 13 – Fichier main.c généré

Remarque : lorsque l'on quitte la perspective Debug, Si une phase de Debug était en cours, celle ci n'est pas stoppée.

Le fichier main.c contient l'ensemble du code généré avec les étapes précédentes. Afin de réaliser cette partie, nous allons modifier ce fichier sans effacer les données générées. Nous pouvons maintenant créer notre premier programme : insérez les lignes suivantes dans le programme existant aux bons endroits :

```

1
2
3  /* variables */
4  int Delay_time = 250;
5
6  /* prototype */
7  void init_GPIO(void);
8
9
10 int main(void)
11 {
12     /* Initialise le uprocesseur et les peripheriques*/
13
14     HAL_Init();
15
16     SystemClock_Config();
17
18     ...
19
20     while(1)
21     {
22
23         // blink LEDs
24         if (GPIO->ODR & 0x0000F000) {
25             GPIO->BSRR |= 0xF0000000;
26         }

```

```

27         else {
28             GPIO->BSRR |= 0x0000F000;
29         }
30         HAL_Delay(Delay_time);
31     }
32 }

```

5.2 Compilation et Debug du projet

Cliquez sur l'icône de compilation du projet (figure 14) ou sur *Project > Build Project*.

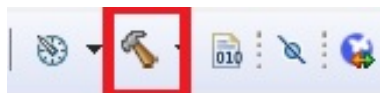


Figure 14 – Compilation du projet

La fenêtre *Console* doit indiquer *Build Finished* sans erreur (pas de lignes rouge). Si ce n'est pas le cas, examinez les erreurs dans cette même fenêtre. Commencez toujours par la première erreur (il peut être nécessaire de remonter le texte de la fenêtre), il arrive souvent que les erreurs suivantes en découlent !

Pour l'instant, notre programme est compilé mais pas encore téléversé sur la carte. Pour effectuer cette action, un programmeur relié en USB au PC est utilisé. Il est présent sur la carte (unité ST-Link V2). Pour réaliser cette action, deux choix s'offre à nous (Boutons figure 15)

- Téléverser en mode Run : Téléverse le programme sur la carte via son programmeur et reset le micro-contrôleur. Ce dernier peut alors exécuter le code et le programmeur est vu comme déconnecté du micro-contrôleur.
- Téléverser en mode Debug : Idem que précédemment mais le programmeur utilise sa fonction de Débogueur pour prendre la main sur le micro-contrôleur. Il peut ainsi figer le microprocesseur (breakpoint) et communiquer avec le PC pour remonter des informations sur son état. C'est la phase de Debug.

Nous allons majoritairement utiliser la deuxième solution dans les TPs, la première étant réservée lorsque le travail est terminé et exempt de toutes erreurs/bug.

Pour lancer la phase de débogage, Cliquez sur l'icône Debug (insecte) ou sur *Run > Debug*.



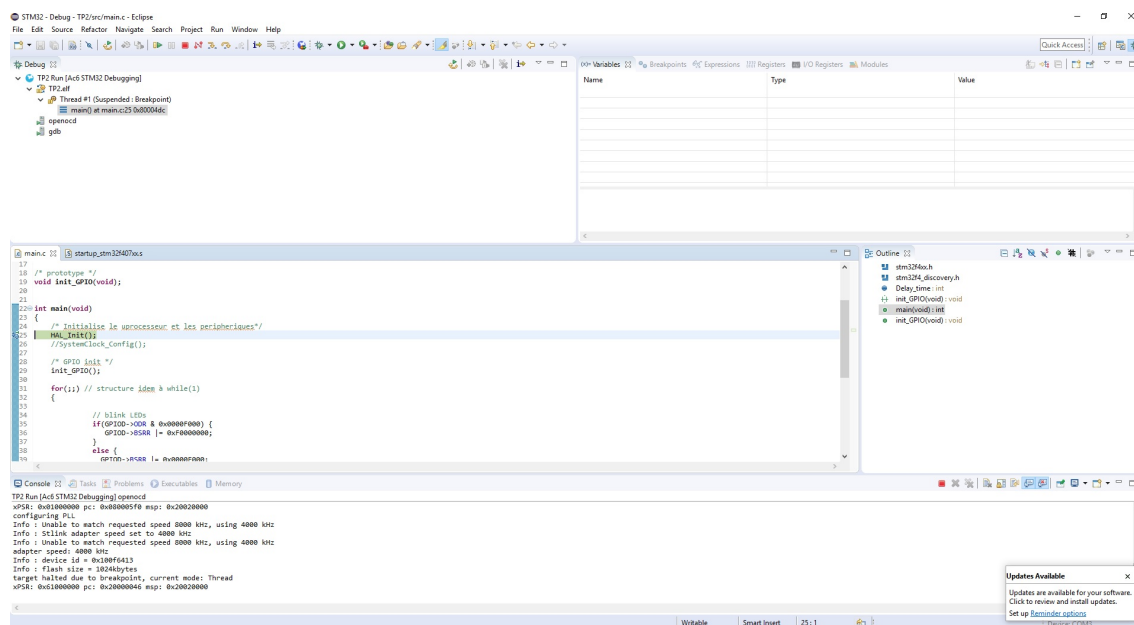
Figure 15 – Choix du mode de programmation

Sur STM32CubeIDE, la première fois qu'une session de Debug est lancée, une fenêtre apparaît vous proposant de paramétrer la configuration de la session de Debug. En SM57, nous utiliserons la configuration par défaut pré-remplie, appuyez donc directement sur OK.

Une fois la programmation réalisée, la session de Debug est lancée et le logiciel change de perspective pour proposer la perspective Debug (voir figure 16).

On retrouve ainsi les fenêtres suivantes :

- File et Outline : Tout comme sur la vue d'édition, permet de modifier le code. Attention ici on utilisera ces fenêtres pour le Debug en mode pas à pas (présenté ci après).

Figure 16 – Fenêtre *Perspective Debug*

- Variables / Breakpoint / Expressions / Registers / SFRs / Modules : Permet la visualisation en temps réel des diverses variables (Expressions) et registres du micro-contrôleur (SFRs).
- Console / Taks / Problems / Executables / Memory (onglets) : Idem que pour la vue Edit, donne un état du micro-contrôleur.

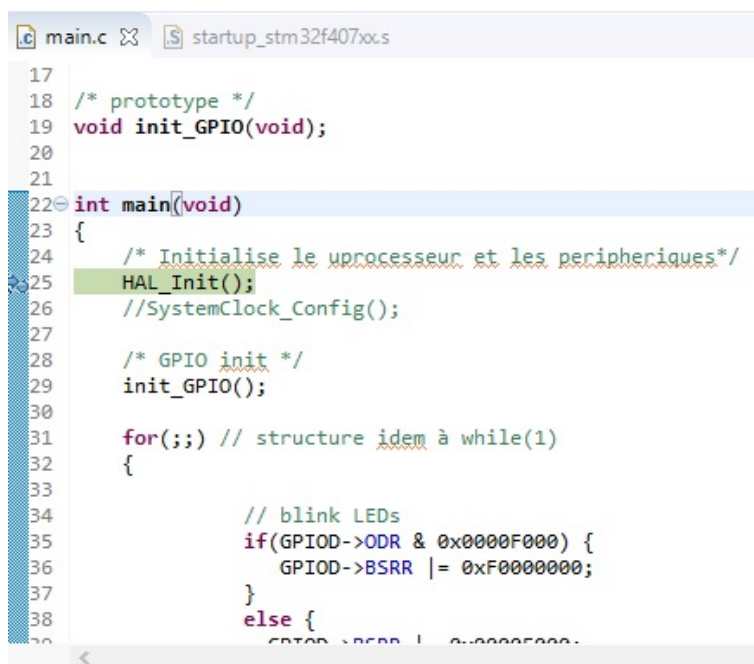
A cette étape, le programme est maintenant chargé dans la carte, le micro-contrôleur à été reset et est figé à la première ligne de notre programme (fonction main). Dans la vue File, nous pouvons voir à quelle ligne le programme est arrêté en regardant la ligne verte (Figure 17). Cette ligne définit la prochaine ligne qui sera lu par le microprocesseur. On remarque également dans la marge une flèche, ayant la même fonction que la ligne verte, mais également un point bleu. Appelé *breakpoint*, il peuvent être placés ou supprimés en double cliquant dans la marge à l'endroit voulu. Ainsi, lorsque le programme sera déroulé en mode *Run*, celui ci s'arrêtera automatiquement arrivé à ce point.

Afin de pouvoir vérifier le bon déroulement du programme, une barre d'outil permet de faire avancer la lecture du microprocesseur du programme (Figure 18)

Les boutons permettent : (de gauche à droite) :

- De lancer le programme en mode continu : celui ci se stoppera automatiquement arrivé au prochain breakpoint.
- De stopper le programme en cours d'exécution (grisé tant que le mode continu n'est pas lancé).
- De stopper la session de Debug, nécessaire avant chaque modification du programme.
- De se déconnecter de la carte (Attention, le mode Debug est toujours actif! A ne pas utiliser).
- D'avancer d'une ligne : Si la prochaine ligne fait appel à une fonction, exécute uniquement la prochaine ligne de la fonction.
- D'avancer d'une ligne : Si la prochaine ligne fait appel à une fonction, exécute uniquement toute la fonction.
- D'avancer jusqu'à sortir de la fonction.

Un autre bouton vous sera utile est celui du reset chip (Figure 19) : Celui ci vous permettra de remettre à 0 le programme sans couper la session de Debug.



```

17
18 /* prototype */
19 void init_GPIO(void);
20
21
22 int main(void)
23 {
24     /* Initialise le uprocesseur et les peripheriques*/
25     HAL_Init();
26     //SystemClock_Config();
27
28     /* GPIO init */
29     init_GPIO();
30
31     for(;;) // structure idem à while(1)
32     {
33
34         // blink LEDs
35         if(GPIOD->ODR & 0x0000F000) {
36             GPIOD->BSRR |= 0xF0000000;
37         }
38         else {
39             GPIOD->BSRR |= 0x0000F000;

```

Figure 17 – Debug du programme

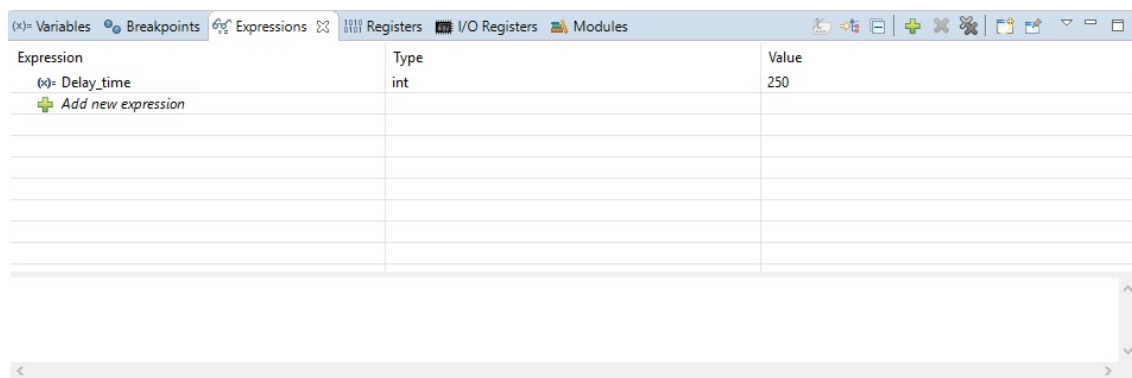


Figure 18 – Menu de Debug



Figure 19 – Bouton reset chip

Afin de pouvoir visualiser les résultats du programme qui seraient "invisibles" sur la carte, les onglets *Expressions*, *Register* et *SFR* nous renseignent sur l'état de ses derniers (figure 20). Il est ainsi possible de vérifier le contenu d'une variable en mémoire ou de l'état d'un registre (interne au microprocesseur ou à un périphérique). Attention : Pour afficher ces variables, le programmeur/débogueur à besoin de figer le microprocesseur. Ainsi, il est impossible de voir ces variables lorsque le mode continu est enclenché.



Expression	Type	Value
Delay_time	int	250
Add new expression		

Figure 20 – Vue d'une variable

5.3 Exercice : Vérification du bon fonctionnement et découverte du programme

Maintenant que nous avons vu tous les outils pour pouvoir déboguer notre programme, nous allons les mettre en oeuvre à travers une série de questions permettant de découvrir son fonctionnement.

1. Si ce n'est déjà fait, lancez une session de Debug du programme et vérifiez que celui-ci soit figé à la première ligne de la fonction `main()` (reset du micro-contrôleur si nécessaire).
2. Lancez le mode continu et observez la carte, que fait le programme ?
3. Appuyez sur le bouton pause. Vous observez qu'un nouveau fichier s'est ouvert dans la fenêtre File : `stm32f4xx_hal.c` et que votre programme est figé dedans. Pourquoi ?
4. Cliquez sur l'onglet `main.c` de la fenêtre File pour revenir à votre programme principal. Double cliquez sur la variable `Delay_time` pour la surligner puis faites un clic droit puis *Add to watch expressions*, cliquez sur *OK*, vous devriez obtenir la même fenêtre que la figure 20. Que signifie *Type* et *Value* ?
5. Double cliquez sur 250 et changez la valeur par 1000 ? Relancez ensuite le mode continu, que s'est-il passé ? À quoi sert donc la fonction `HAL_Delay()` ? (vous pouvez essayer plusieurs valeurs différentes).
6. Placez un breakpoint à la ligne : `if(GPIOD->ODR & 0x0000F000)` (double clic dans la zone bleue) Pourquoi la ligne devient-elle verte ?
7. Cliquez sur l'onglet *I/O Registers* puis sur *GPIO* puis étendez *GPIOD* ; enfin double cliquez sur *GPIOD* pour que celui-ci et tous les registres dépendant deviennent verts. On remarque que les champs se remplissent. À quoi correspondent-ils ?
8. Cherchez le registre *ODR* puis cliquez sur son champ *Binary Value*. Un mini menu apparaît avec des cases grises vous permettant de modifier chacun des bits du registre en cliquant dessus. Inversez la valeur des bits : 15 14 13 12 puis appuyez sur *Set* (tout à droite du menu). Visualisez l'effet sur la carte. Recommencez si besoin. Que s'est-il passé ?
9. Vous trouverez sur moodle dans *IF70 -> Documentations Techniques -> Schéma Carte STM32F407G-DISC1* le schéma électrique de la carte, vérifiez les sorties PD15, PD14, PD13 et PD12. À quoi sont-elles physiquement reliées ? Que pouvez-vous en conclure ?
10. Figez le microprocesseur sur la ligne où vous aviez placé votre breakpoint. Appuyez sur *F5* (step into) 2 fois (visualisez l'effet d'un appui sur le code) et regardez le registre *IDR*. Que s'est-il passé ?
11. Lancez sur le mode continu pour revenir au breakpoint, refaites l'opération précédente. Concluez sur la fonctionnalité réalisée par le programme.

Dans cette section, nous allons analyser le projet généré afin de comprendre le fonctionnement de la librairie HAL (Hardware Abstraction Layer) pour pouvoir ensuite l'utiliser afin de programmer en langage C.

6 Analyse du projet STM32CubeIDE généré

Pour cette partie, reprenez le fichier `main.c` original généré que vous aviez copié/collé dans un fichier texte.

Commencez par ouvrir le fichier **main.h** (*ctrl+click* sur `main.h` dans `main.c` ouvre automatiquement le fichier. Autre solution : Dans project explorer, `main.h` est rangé dans le dossier Inc) :

1. A partir de la ligne 60, vous trouverez une série de *define*, à quoi correspondent ils ?
Chaque broche utilisée possède un numéro de broche (pin) ainsi qu'un port (de A à E).
Exemple : la broche PA2 est la broche 2 du port A.
Ouvrez maintenant le fichier **main.c** :
2. Qu'est il fait dans la fonction *in main(void)* ?
3. Allez à la fonction *static void MX_GPIO_Init(void)* : Quelle est le rôle de cette fonction ?
4. Même question pour la fonction *void SystemClock_Config(void)*

Observons maintenant la librairie HAL et notamment sa partie concernant les entrées sortie afin de voir les fonctions qu'elle offre. Ouvrez le fichier **stm32f4xx_hal_gpio.h** situé dans *Drivers -> STM32F4xx_HAL_Driver -> Inc.* Allez ensuite à la ligne 234. Vous trouverez les prototypes des fonctions suivantes :

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
void HAL_GPIO_WritePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

1. Pour chacune d'entre elle, déterminer leur fonctionnement (vous pouvez voir le code source de la fonction en faisant un ctrl+click sur le nom de la fonction).
2. Cherchez la déclaration de *GPIO_PinState*. Quels sont les deux valeurs possible que ce type de variable pour avoir ? vous pouvez utiliser ces valeurs comme 0 ou 1 (en les écrivant en toute lettre dans votre code).

6.1 Programmation avec la librairie HAL

Essayons maintenant de créer un programme en utilisant ces fonctions. Vous écrirez tous vos programmes dans la fonction main. Pour chaque modification, si vous souhaitez conserver les réponses aux questions précédentes, vous pourrez mettre les lignes de code en commentaire (// ou /* puis */) :

1. Écrivez un code qui permet d'allumer la led verte puis de l'éteindre après 1 seconde.
2. Écrivez un code qui permet d'allumer de faire clignoter toutes les Led avec une période de 200 ms. Utilisez la fonction toggle pour ceci.
3. Écrivez un code qui permet de lire la valeur du bouton poussoir et de renseigner une variable à 1 ou 0 si le bouton est appuyé ou non. Vérifiez le bon fonctionnement en déboguant (voir TP2).

Aide : N'oubliez pas de déclarer une variable de type *GPIO_PinState* Exemple :

```
GPIO_PinState Pbutton_state;
```

6.2 Exercice : Chenillard

Le chenillard est un grand classique de la découverte des microcontrôleurs !



1. Il vous faut donc réaliser un chenillard sur 4 LEDs. Vous êtes libres d'utiliser la méthode algorithmique qui vous convient mais des points supplémentaires seront attribués à ceux qui auront su faire preuve de finesse !
2. Modifiez le code pour avoir la possibilité de changer la vitesse du chenillard lorsqu'on appuie sur un bouton poussoir.

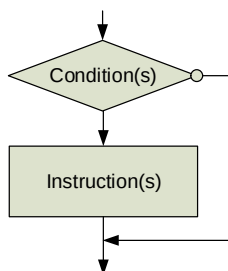
TP N°2- Programmation en C

Ce TP vous permettra de vous familiariser avec le langage C. Vous utiliserez le projet créé au TP précédent pour réaliser les exercices.

1 Branchements conditionnels

Les branchements conditionnels permettent d'exécuter une ou plusieurs instructions séquentiellement si les conditions déterminées sont remplies.

1. Une structure conditionnelle de type SI...ALORS respecte cet organigramme et s'écrit de la façon suivante.



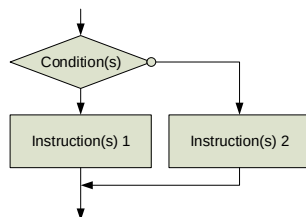
```

1      if (condition) instruction;    // pour une seule instruction
2
3
4      if (condition)                 // pour un bloc d'instructions
5      {
6          instruction1;
7          instruction2;
8      }
  
```

Écrivez un code en langage C qui permet d'allumer la diode rouge lorsqu'on appuie sur le bouton poussoir.

N.B. : Pensez à insérer votre code dans une boucle infinie (`while(1){ ... votre code ... }`)

2. Une structure conditionnelle de type SI...ALORS...SINON respecte cet organigramme et s'écrit de la façon suivante.



```

1      if (condition) instruction1;    // pour une seule instruction par
2      condition
3
4
5      if (condition)                  // pour un bloc d'instructions par
6      condition
7      {
8          instruction1;
9          instruction2;
10         ...
11     }
12     else
13     {
14         instructions3;
15         instructions4;
16         ...
17     }

```

Écrivez un code en langage C qui permet d'allumer la diode rouge lorsqu'on appuie sur le bouton poussoir et de l'éteindre si non.

3. Une structure conditionnelle de type SWITCH...CASE permet la comparaison d'une expression (ce peut être une simple variable) avec une multitude de valeur s'écrit de la façon suivante.

```

1  switch (identificateur)
2  {
3      case valeur1 :    instruction1;
4                      break;
5      case valeur2 :    instruction2;
6                      break;
7      ...
8      case valeurN-1 :  instructionN-1;
9                      break;
10     default :         instructionN;
11                     break;
12 }

```

Écrivez un code en langage C qui permet de réaliser la table de vérité suivante (OFF pour bouton relâché ou diode éteinte, ON pour bouton appuyé ou diode allumée) :

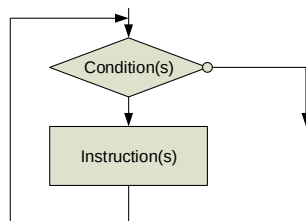
Bouton poussoir	Variable toto	Diode verte	Diode orange	Diode rouge	Diode bleue
OFF	0	OFF	OFF	OFF	ON
OFF	1	OFF	ON	ON	OFF
ON	0	ON	OFF	ON	OFF
ON	1	OFF	ON	OFF	OFF

Pour réaliser cette fonction, Déclarer une variable toto en variable globale. Pour vérifier son bon fonctionnement, vous utiliserez le débogueur de forçant la valeur de toto à 0 ou à 1 (voir TP2).

2 Boucles

Les boucles permettent de réaliser un nombre d'itérations déterminé à l'avance ou non d'un même bloc d'instructions.

1. Une structure de boucle TANT QUE ... FAIRE respecte cet organigramme et s'écrit de la façon suivante.



```

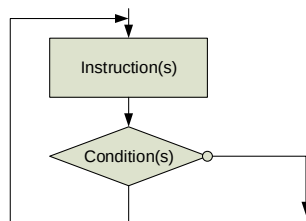
1      while (condition)  instruction;    // pour une seule instruction
2
3
4      while (condition)                                // pour un groupe d'
5      {
6          instruction1;
7          instruction2;
8          ...
9      }
  
```

La *condition* consiste généralement en une comparaison (test d'égalité, d'inégalité, de supériorité ou d'infériorité). Voir la partie du cours sur les opérateurs relationnels. Le résultat d'une comparaison est un booléen, c'est à dire qu'il est soit vrai (valeur 1) soit faux (valeur 0). Ne confondez pas l'opérateur d'assignation = et l'opérateur de test d'égalité ==.

Vous savez déjà écrire une boucle infinie (avec *while(1)*). Maintenant, vous allez écrire une boucle qui permet de réaliser une temporisation entre l'allumage et l'extinction d'une des diodes. Voici quelques indications pour créer une boucle de temporisation :

- déclarez une variable pouvant contenir la valeur la plus élevée possible.
- initialisez cette variable à 0.
- dans une boucle, tant que cette variable n'est pas égale à une valeur que vous déterminerez, vous incrémenterez cette variable
- une fois sorti de la boucle car la condition est rempli, remettez à 0 la variable.

2. Réalisez une temporisation similaire à la précédente, mais cette fois ci en utilisant une structure du type FAIRE ... TANT QUE.



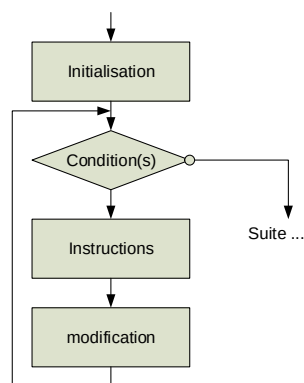
```

1      do instruction;           // pour une seule instruction
2      while (condition);
3
4
5      do                       // pour un groupe d'instructions
6      {
7          instruction1;
8          instruction2;
9          ...
10     } while (condition);

```

Quelle est la différence majeure avec la structure précédente ?

- Une structure de boucle POUR...FAIRE...JUSQUA respecte cet organigramme et s'écrit de la façon suivante.



```

1      for (initialisation; condition de continuité; modification)
2      {
3          instruction1;
4          instruction2;
5          ...
6      }

```

Écrivez une fonction qui réalise le calcul de la fonction mathématique factorielle. La variable d'entrée sera notée n et le résultat sera stocké dans une variable res sur 16 bits (d'où $res = !n$).

Cette fonction utilisera une boucle *pour*. Choisissez judicieusement la taille de codage de n et déclarer donc cette variable en fonction de ce critère (justifiez votre choix). Quelle sera la valeur maximale de n pour laquelle le résultat sera encore correct ?

Vous utiliserez le débogueur pour vérifier le fonctionnement de votre code.

3 Création de fonctions ou sous-programmes

Comme vous le savez maintenant, votre programme se compose d'au minimum un programme principal (ou fonction principale), le *main*. A ce programme principal peut s'ajouter des sous-programmes permettant de structurer votre code. Ces sous programmes seront appelés par le programme principale ou bien par d'autres sous-programme. Nous allons maintenant apprendre à créer ces sous-programmes.

- Créez une fonction *tempo* qui reprend le code de la temporisation créée avec une boucle plus haut. Quel sera le type de cette fonction ? Doit-elle prendre des paramètres en argument ?

Réécrivez alors le code du clignotement de la diode afin d'utiliser cette fonction. Quelle est la différence entre les deux programmes ?

2. Ecrivez une fonction *factorielle* qui calcule la factorielle d'un nombre n . Cette fonction prend en paramètre le nombre n et renvoie la valeur du calcul. De quel type sera la fonction ? Utilisez cette fonction dans le programme principal et tester l'ensemble du code avec le débogueur.

TP N°3- Mini projet : Dispositif antivol de la carte STM32F407

Ce TP met en œuvre différents composants et périphériques de la carte STM32F407-DISCOVERY pour la création d'un projet d'antivol de la carte par détection de mouvement. Ce TP sera découpé en 2 séances.

1 Introduction

1.1 Objectifs

L'objectif de ce TP est d'utiliser l'accéléromètre, le convertisseur digital analogique et l'horloge temps réel présent sur la carte pour réaliser un dispositif de détection de mouvement de la carte. Pour ce faire, le groupe de TP sera réparti en 3 sous groupe qui devront collaborer entre eux afin d'étudier un composant spécifique du système. Ensuite, ils devront exposer les compétences acquises lors d'une soutenance de milieu de projet afin que les 2 autres groupes soit capables de comprendre et d'utiliser leur code pour réaliser le programme final.

1.2 Cahier des charges

Le programme devra réaliser les fonctions suivantes :

- L'accéléromètre doit être configuré pour détecter un mouvement significatif (quelqu'un s'empare de la carte)
- Un signal est émis et l'heure à laquelle le mouvement est détecté s'affiche sur le terminal (liaison série)
- La carte émet un signal audio (beep) via sa sortie audio jack.

Attention : Ce TP étant défini comme un projet, vous ne trouverez que peu d'informations ou questions dans l'énoncé. Il appartient à votre équipe d'identifier les différentes étapes à réaliser afin de valider la fonction demandée, et également de pouvoir restituer les connaissances acquises aux autres membres du groupe de TP.

1.3 Notation et bonus

Durant ce TP vous serez évalué sur les critères suivants :

- Une soutenance de milieu de projet vous sera imposée : vous devrez présenter à l'ensemble du groupe les résultats obtenus et les codes développés : vous devrez présenter cette soutenance comme un cours afin que les autres étudiants puisse comprendre et réutiliser votre programme pour le projet complet.
- Un rapport final présentant le projet dans sa globalité sera effectué. Ainsi que les codes sources du projet.
- Une partie de la notation sera dédié à l'utilisation de la HAL et de bibliothèques présente sur internet. Moins vous utiliserez ces bibliothèques, et plus votre note sera élevée !

2 RTC : Real Time Clock

Afin de traiter cette partie, vous trouverez sur Moodle et Teams dans la datasheet complète du STM32F407 : *Family Reference*, une section spécifique (page 798) sur le périphérique Real-time clock :

1. Étudiez le fonctionnement du module RTC du micro-contrôleur et ses registres, et proposez une fonction qui permet d'initialiser l'horloge. Dans un premier temps, utilisez les fonctions HAL disponibles ainsi que CubeMX afin de créer une fonction d'initialisation via l'interface graphique.
2. Vous trouverez sur Moodle dans *SM57->Projets STM32CubeIDE et librairies pour les TPS -> Librairie UART* un fichier source *uart2.c* et entête *uart2.h* vous permettant de communiquer avec le PC (il vous faudra installer un terminal de liaison série comme par exemple *Putty*). Analyser la librairie et utilisez la afin d'afficher l'heure et le date de la RTC sur le terminal.
3. Améliorez votre programme en utilisant le bouton poussoir de la carte par interruption pour afficher l'heure sur le terminal à chaque appui.
4. Optimisez votre programme en remplaçant les fonctions HAL par des fonctions propres que vous créez vous même.

3 Accéléromètre LIS3DSH

Afin de traiter cette partie, vous trouverez sur Moodle et Teams la datasheet complète du composant : *LIS3DSH*, ainsi que le schéma électrique de la carte DISCOVERY.

Attention : Vous pouvez trouver dans les différentes datasheet ou sur STM32CubeIDE la référence LIS302DL, il s'agit dans ce cas de notre composant, le LIS3DSH (certaines version de la carte DISCOVERY sont équipées avec un accéléromètre de type : LIS302DL).

1. Étudiez le fonctionnement du composant LIS3DSH et de sa connexion au micro-contrôleur. Analysez ces registres afin de comprendre comment paramétrer le composant via la liaison SPI ainsi que pouvoir récupérer ces données. Dans un premier temps, vous pourrez utiliser CubeMX en initialisant par défaut les GPIOs de la carte et la HAL afin de pouvoir communiquer facilement avec le composant.
Afin de vérifier que la communication fonctionne, vous pouvez dans un premier temps lire le registre WHO_AM_I.
2. Vous trouverez sur Moodle dans *SM57->Projets STM32CubeIDE et librairies pour les TPS -> Librairie UART* un fichier source *uart2.c* et entête *uart2.h* vous permettant de communiquer avec le PC (il vous faudra installer un terminal de liaison série comme par exemple *Putty*). Afficher les données XYZ de l'accéléromètre afin de valider la bonne acquisition de celles ci.
Attention : Pour lire les données XYZ, il faut d'abord lire 2 fois dans le composant (lire par exemple le registre de contrôle CTRL_REG5).
3. Vous remarquerez avec le schéma électrique qu'une sortie du LIS3DSH est reliée à une entrée du micro-contrôleur en interruption. Utilisez cette fonctionnalité pour déclencher une interruption dans le STM32F407 lorsque vous détectez un mouvement avec l'accéléromètre. On pourra utiliser l'USART2 pour afficher un message sur le terminal quand l'interruption se déclenche.
4. Vous trouverez sur Moodle dans *SM57->Projets STM32CubeIDE et librairies pour les TPS -> Librairie SPI* un fichier source *spi.c* et entête *spi.h*. Optimisez votre code en utilisant cette librairie dans votre projet au lieu de la HAL et des fonctions d'initialisation générées par CubeMX pour communiquer en SPI avec le composant.
5. Optimisez votre programme en remplaçant les fonctions HAL par des fonctions propres que vous créez vous même.

4 Convertisseur Digital Analogique : CS43L22

Afin de traiter cette partie, vous trouverez sur Moodle et Teams la datasheet complète du composant : *CS43L22*, ainsi que le schéma électrique de la carte DISCOVERY et sa datasheet (utile pour comprendre quels composant sont reliés au CS43L22).

1. Étudiez le fonctionnement du composant CS43L22 et de sa connexion au micro-contrôleur. Analysez ces registres afin de comprendre comment paramétrer le composant via la liaison SPI ainsi que pouvoir émettre un beep sur la sortie Jack. Dans un premier temps, vous pourrez utiliser CubeMX en initialisant par défaut les GPIOs de la carte et la HAL afin de pouvoir communiquer facilement avec le composant.
2. Vous trouverez sur Moodle dans *SM57->Projets STM32CubeIDE et librairies pour les TPS -> Librairie UART* un fichier source *uart2.c* et entête *uart2.h* vous permettant de communiquer avec le PC (il vous faudra installer un terminal de liaison série comme par exemple *Putty*). Vous pouvez afficher à l'écran la validation des différentes trames que vous envoyées en I2C pour vous aider à déboguer.
3. Optimisez votre programme en utilisant le bouton poussoir pour changer la fréquence de Beep émis par le circuit sur la sortie jack. vous pouvez également rentrer la fréquence du Beep voulu sur l'USART.
4. Vous trouverez sur Moodle dans *SM57->Projets STM32CubeIDE et librairies pour les TPS -> Librairie I2C* un fichier source *I2C.c* et entête *I2C.h*. Optimisez votre code en utilisant cette librairie dans votre projet au lieu de la HAL et des fonctions d'initialisation générées par CubeMX pour communiquer en SPI avec le composant.
5. Optimisez votre programme en remplaçant les fonctions HAL par des fonctions propres que vous créez vous même.

5 Programme final

Une fois votre composant étudié et validé, récupérez chez vos camarades les codes sources de chacune de leur partie et assistez aux présentations qu'ils feront. A partir de la, crée le projet final consistant à assembler les trois composants ensemble pour réaliser la fonction final d'antivol.

Vous êtes libre ici de choisir comment et quoi faire avec les composant pour réaliser le programme final (gestion par interruption ou non), des points supplémentaire seront attribué aux idée original mettant en ?uvre des fonctions annexes des composants.

TP N°4- Programmation d'une bibliothèque CAN pour le STM32F407

1 Objectifs et rendus

Ce TP sera découpé sur 2 séances. Il fera l'objet d'un compte rendu noté sur Moodle à remettre une semaine après la dernière séance de ce TP.

Ce TP est fait afin de pouvoir construire une bibliothèque qui configure et utilise le bus CAN. Le but de cette brique logicielle est qu'elle puisse être ensuite utilisée dans le contexte du projet.

Ce TP s'appuie largement sur le TD vu en cours dont vous retrouverez la vidéo sur Youtube, n'hésitez pas à reprendre les codes et principes théoriques vus en séance.

Vous trouverez sur Moodle dans *SM57 -> Datasheet des composants utilisés -> Datasheet STM32F407G* la documentation concernant le module CAN du STM32F407 afin de réaliser le TP. Vous trouverez également dans *SM57 -> Cours -> Module CAN STM32F407* un document présentant le module BxCAN avec tous ces registres et des exemples de fonctionnement. Analysez ces documents avant de commencer à répondre aux questions du TP.

Vous trouverez également sur la chaîne Youtube plusieurs vidéos sur le module BxCAN qui vous aiderons.

Pour traiter ce TP, vous créerez un fichier *can.h* et *can.c* dans lesquels vous traiterez les questions. N'oubliez pas également de définir des variables globales si vous avez besoin de lier des variables entre la librairie CAN et le fichier *main.c*.

Même si la bibliothèque finale sera réalisée sans utilisation (ou hors fonction CAN) de la HAL, je vous conseille dans un premier temps de répondre à l'ensemble des questions en utilisant la HAL pour bien comprendre le fonctionnement du module BxCAN avant de les remplacer par vos propres fonctions.

2 Configuration du module BxCAN du STM32F407

1. A partir des questions vues en TD, créez une fonction *CAN_config* qui configure le module CAN pour répondre aux caractéristiques suivantes :
 - CAN 2.0
 - Vitesse de communication de 250kbits/s.
 - Génération d'une interruption lors de la réception d'une trame dans la boîte de réception *FIFO0*.
 - Mode de fonctionnement : Loopback.
 - Segment de Phase 1 BT1 de $6t_q$.
 - Segment de Phase 2 de BT2 $3t_q$.
 - saut de resynchronisation de $1t_q$.

2. Modifiez ensuite cette fonction pour y ajouter des paramètres d'entrée afin de pouvoir configurer les filtres de réception de trame comme l'utilisateur le souhaite. On pourra par exemple prendre le prototype suivant (le bit IDE définit si les filtres seront en standard ou étendu) :

```
void CAN_config(uint8_t IDE, uint16_t Filter_ID_high, uint16_t Filter_ID_low,
                uint16_t Filter_Mask_high, uint16_t Filter_mask_low);
```

Si vous le désirez, vous pouvez également rajouter des arguments (exemple : utilisation des filtres d'acceptation en mode mask ou list ect...)

3. **Pour cette question, vous aurez besoin d'être dans la salle de TP ou sur l'ordinateur du professeur afin de la valider ou d'avoir terminé la partie 3.**

Réalisez un exemple d'appel de la fonction pour ne recevoir uniquement que la trame d'identifiant 0x010 dans FIFO0. Vérifiez son bon fonctionnement avec le débogueur en validant l'entrée dans l'interruption (prendre des impressions d'écran pour le rapport). On pourra également allumer une LED lorsque l'interruption de réception est déclenchée.

4. **Pour cette question, vous aurez besoin d'être dans la salle de TP ou sur l'ordinateur du professeur afin de la valider ou d'avoir terminé la partie 3.**

Refaites la question en simulant maintenant un filtre avec une trame étendue de votre choix. Vérifiez en simulation.

3 Création d'une fonction d'écriture sur le bus

Ayant précédemment configuré le module CAN en mode loopback, pour éviter tout bugs, désactivez l'interruption de réception de *RX0* dans la configuration du module CAN temporairement pour traiter cette partie.

1. Créez une structure de donnée intitulée *CAN_frame* dans *can.h* comme vu en TD afin de pouvoir manipuler des trames dans les futures questions. Ici la structure pourra prendre des champs supplémentaires afin de pouvoir construire soit une trame d'identifiant standard, soit étendu. Ajoutez également un champ *RTR* avec de pouvoir définir des trames de requête ou de donnée.
2. A partir des questions traitées en TD, réalisez une fonction d'écriture d'une trame dont le prototype sera :

```
unsigned uint8_t envoi_CAN(CAN_frame CAN_mes)
```

La fonction renverra 1 si la trame a bien été traitée et 0 sinon.

3. Modifiez votre projet pour mettre en place une interruption sur le bouton poussoir de la carte *STM32F407 DISCOVERY*.
4. Écrivez la fonction d'interruption afin d'envoyer une trame à chaque appui du bouton poussoir avec les caractéristiques suivantes :
 - Trame de donnée
 - Identifiant 0x010
 - 1 octet de donnée
 - La donnée commence à 0 et s'incrémente de 1 à chaque fois que la trame est émise
5. **Pour cette question, vous aurez besoin d'être dans la salle de TP ou sur l'ordinateur du professeur afin de la valider.** A l'aide de l'outil CAN Analyser, vérifiez le bon envoi de la trame.

4 Réception par interruption d'une trame

1. Afin de pouvoir faire le lien entre les trames reçus par interruption et les trames à traiter dans le programme principale, créez un buffer circulaire avec ces variables de gestion d'écriture et de lecture. On pourra prendre une taille de buffer de 10 éléments.
2. Écrivez la fonction d'interruption du module CAN pour la réception de la boîte *RX0* dont le prototype sera :

```
void CAN1_RX0_IRQHandler(void)
```

3. Utilisez le programme d'envoi avec l'appui du bouton poussoir réalisé dans la partie précédente afin de tester votre fonction d'interruption en mettant un breakpoint et en regardant avec l'outil *Variables* l'état des registres et le bon remplissage du buffer circulaire.
4. Utilisez la librairie *USART2* disponible sur Moodle lors du TP précédent afin d'afficher sur un terminal tous les éléments concernant la trame reçue :
 - Type de trame.
 - Identifiant.
 - Données.

Vous aurez, pour ce faire, besoin de créer une fonction qui convertit un nombre entier en caractère. Vous trouverez cette fonction dans la librairie.

5 Mode de fonctionnement requête/réponse

Pour répondre à cette partie, on pourra créer les fichiers *req_resp.h* et *req_resp.c* afin de ne pas surcharger le programme principal et pouvoir réutiliser ensuite votre projet STM32CubeIDE pour d'autres fonctionnalités.

1. Réalisez un programme qui, lors de l'envoi du message : *"trame"* provenant de l'UART, génère une trame de requête d'identifiant 0x010.
2. Réalisez un programme qui, lors de la réception d'une trame de requête d'identifiant 0x010, envoie une trame de donnée correspondant avec pour donnée l'état des 4 leds de la carte *STM32F407*.
3. Tester votre programme (on pourra faire varier l'état des dans le main pour rendre le test plus ludique).
4. Écrivez un programme qui envoie sur l'UART un message donnant l'état des leds (*"led allumée"* ou *"led éteinte"*) à la réception de la trame de donnée.
5. Testez votre programme final.
6. Une fois toutes les questions traitées et validées en, loopback, vous pourrez essayer en mode normal avec l'outil CAN Analyser disponible en salle de TP.

TP N°5- Programmation d'une communication LIN

1 Objectifs et rendus

Ce TP sera découpé sur 2 séances. Il fera l'objet d'un compte rendu noté sur Moodle à remettre une semaine après la dernière séance de ce TP.

Ce TP est fait afin de pouvoir construire une bibliothèque qui configure et utilise le bus LIN basé sur l'USART déjà vu lors de TP précédents. Le but de cette brique logicielle est qu'elle puisse être ensuite utilisée dans le contexte du projet.

Vous trouverez sur Moodle dans *SM57 -> Cours -> Module CAN STM32F407* la documentation concernant le module USART du STM32 afin de réaliser le TP, et bien évidemment plus de détails dans la datasheet complète *Family Reference*

Pour traiter ce TP, vous trouverez sur Moodle les fichiers *UART_LIN.h* et *UART_LIN.c* vides dans lesquels vous traiterez les questions. N'oubliez pas également de définir les variables globales qui sont utilisées en extern dans le fichier header dans le fichier *main.c*.

Afin de valider les questions demandées, je vous conseille fortement d'utiliser également l'USART2 dont vous calquerez exactement le code réalisé pour USART3, ceci vous permettra à l'aide d'un terminal de valider le fonctionnement de chaque étape. On pourra pour se faire utiliser les directives `#if` `#ifndef` un mode debug où USART2 sera la copie de USART3 (voir vidéo sur les directives C).

2 Création de la bibliothèque LIN du dsPIC30F4011

Attention : Le protocole LIN que nous allons développer utilise le module USART. Cependant, les octets envoyés seront sous forme numérique et non des caractères. L'hyperterminal (Putty) connecté au module USART que vous allez utiliser ne permettra pas d'afficher les résultats escomptés correctement.

Pour chaque question, vous trouverez le squelette de la fonction demandée dans le fichier *UART_LIN.c*

1. Créez la fonction *UART_Init* qui configure le module USART3 pour répondre au caractéristiques suivantes :
 - Communication bidirectionnelle
 - Vitesse de communication de *9600bauds*.
 - Utilisation des broches TX et RX en fonction du schéma électrique de la carte fille.
 - 8 bits de donnée, 1 bit de stop, pas de parité.
 - Activation d'aucune interruption (vous modifierez ce paramètre au fur et à mesure de votre avancée du TP).
2. Écrivez la fonction *Uart1_PutChar* qui envoi l'octet donnée en paramètre sans utiliser d'interruption. Testez cette fonction en envoyant un caractère sur l'USART3.
3. Créez la fonction *sync_break* qui :
 - Envoi 10 bits de break
 - Envoi ensuite le champs de synchronisation.
 Testez la fonction avec un terminal, Que constatez vous ?

4. Dans le fichier *UART_LIN.c* vous trouverez la fonction *checksum* déjà implémentée. Analysez cette fonction : Comment le calcul du checksum fonctionne il ? En utilisant la structure *LINMSG* définie dans *UART_LIN.h*, comment intégrer le checksum dans cette structure ? Donnez un exemple. Donnez également un exemple d'utilisation de la fonction checksum dans le cas où l'on veut vérifier un checksum reçu par un esclave.

2.1 Envoi d'un message complet (écriture sur un esclave)

5. Nous allons dans cette question réaliser l'envoi d'un message complet (cas où le maître envoie le Message Header et le Message Response) Écrivez la fonction *SendMessage* qui prend en paramètre une variable de type *LINMSG* et envoie la trame complète à l'esclave. Vous vous aiderez des fonctions réalisées précédemment. Si vous le souhaitez, vous pouvez découper la fonction avec l'appel à deux sous fonctions : *SendHeader* et *SendResponse*. Testez la fonction en envoyant une trame toutes les secondes et en vérifiant avec le terminal.

2.2 Envoi d'un message Header et réception de la réponse (lecture sur un esclave)

Nous allons dans cette question réaliser l'envoi d'un message Header, puis recevoir par interruption la réponse. Pour ce faire, vous activerez les interruptions par réception de l'USART au moment opportun puis les désactiverez lorsque celles-ci ne seront plus nécessaires.

6. Écrivez la fonction *SendRequest* qui prend en paramètre une variable de type *LINMSG* et envoie le message Header.
7. Écrivez la fonction *slave_response* qui est appelée depuis la fonction d'interruption de réception d'un octet et qui réalise la réception octet par octet de la trame LIN de réponse. Pour tester cette fonction, vous pouvez utiliser la maquette à distance proposée par le professeur.

3 Échange entre 2 cartes via le protocole LIN

Dans cette partie, vous allez communiquer avec un autre camarade via le protocole LIN : Le protocole n'autorisant qu'un seul maître à la fois, vous réaliserez 2 projets, un maître et un esclave suivant la configuration que vous souhaitez tester avec votre binôme partenaire. En séance, vous connecterez les 2 cartes avec un câble DB9 pour valider le fonctionnement de vos programmes.

1. En discussion avec votre partenaire, choisissez chacun une adresse esclave de lecture et une d'écriture.
2. En reprenant les bibliothèques et fonctions des précédents TPs : Intégrez les fonctions utiles pour récupérer les données de l'accéléromètre et de la RTC de la carte. Un des membres du binôme prendra l'accéléromètre et l'autre la RTC. Intégrez également la bibliothèque CAN réalisée dans le TP précédent.
3. Réalisez un programme qui, après la détection de votre PID LIN d'esclave d'écriture, répond avec une trame donnant la valeur d'un des axes de l'accéléromètre ou de la date et heure provenant de la RTC (vous devrez peut-être enrichir la bibliothèque LIN de nouvelles fonctions propres au fonctionnement esclave).
4. Écrivez un programme qui, après la détection de votre PID LIN d'esclave de lecture, envoie la donnée reçue en LIN sur le bus CAN via une trame de donnée dont vous choisirez l'identifiant librement.

5. Échangez avec votre binôme partenaire afin de créer un programme final qui, pour l'un, demande la valeur de l'accéléromètre et la renvoi sur le bus CAN lors de la réception et pour l'autre, demande la valeur de la RTC et la renvoi sur le bus CAN également.
6. Refaire de même en intervertissant les rôles, le groupe qui était maitre devient esclave et inversement.

TP N°6- Interaction avec la maquette Projet

1 Objectifs et rendus

Ce TP se déroulera sur une séance. Ce TP sera noté sous la forme d'une soutenance de 10 minutes lors de la prochaine séance de TP.

Dans ce TP, vous allez mettre en oeuvre les deux bibliothèques LIN et CAN réalisées précédemment pour interagir avec la maquette (CAN) ainsi qu'un autre binôme de TP (LIN).

Vous trouverez sur Moodle dans *SM57 -> Datasheet des composants utilisés -> Maquette SM57* la documentation concernant la maquette CAN utilisée pour le projet.

Pour traiter ce TP, vous allez vous associer à un autre binôme et chacun choisira un composant de la maquette CAN à instrumenter. Faites attention à ce qu'un binôme prenne un composant constitué majoritairement de capteurs (comodos, satellite, portière) et un autre d'actionneurs (phares avant, arrière).

Un simulateur de la maquette est présent sur le PC à distance et vous permettra de tester votre programme avec l'envoi/réception des différentes trames CAN inhérentes à la maquette. Des vidéos Youtube sont à disposition pour vous montrer des exemples.

2 Instrumentation d'un élément de la maquette CAN

Après avoir choisi le composant que vous souhaitez utiliser : Pour implémenter votre code concernant spécifiquement le composant, vous créez les fichiers *NOM_composant.h* et *NOM_composant.c*.

Parcourez sa description et interprétez toutes les trames CAN nécessaires à la communication avec le composant. Vous regrouperez ces trames dans un fichier Excel qui pourra être présenté au professeur.

1. Configurez le module CAN du STM32F407 pour ne recevoir uniquement que les trames voulues par le composant. Vous pouvez utiliser les deux FIFO si vous le désirez.
2. Réalisez un programme qui récupère l'ensemble de l'état de chaque fonction du composant et les stocks dans des variables par un système de requête/réponse.
3. Si vous instrumentez un composant actionneur : Réalisez un programme qui permet de changer l'état de chaque actionneur. Si vous instrumentez un composant capteur : Adaptez vos programmes pour qu'ils soient compatibles avec une réception par impulsion.

3 Communication avec un autre binôme en LIN)

Maintenant que vous pouvez communiquer avec votre composant CAN, vous allez mettre en place une communication LIN avec un autre binôme afin d'avoir accès à un deuxième composant à travers lui. Pour implémenter votre code concernant cette communication, vous créez les fichiers *COM_LIN.h* et *COM_LIN.c*.

1. En concertation avec un deuxième binôme, établissez un protocole d'échange de messages LIN afin d'établir une interaction entre les 2 composants CAN qui sont instrumentés (exemple : Binôme 1 demande l'état du bouton lève vitre de la portière, binôme 2 répond. Si celui-ci est à 1, alors il allume les feux de croisement du phare avant).
Vous regrouperez dans un fichier Excel la composition des trames LIN échangées (PID,

nombre de données, valeurs...) que vous pourrez présenter au professeur. Ayez au moins 5 interactions différentes (vous pouvez en faire beaucoup plus avec le nombre de capteur/actionneur présent sur chaque composant).

BONUS : Ne vous limitez pas aux interactions avec la maquette, soyez créatif et utilisez les autres composants de la carte (RTC, Beep, Accéléromètre) pour faire des fonctions originales !

2. Implémentez la communication LIN pour réaliser les interactions, vous pourrez utiliser la maquette de PC à distance pour simuler l'envoi et la réception des trames LIN avec le simulateur STM32 si votre binôme partenaire n'est pas encore assez avancé dans sa partie.

TP N°7- Intégration du système d'exploitation FreeRTOS dans le projet et optimisation de la consommation énergétique

1 Objectifs et rendus

Ce TP se déroulera sur 3 séances. Ce TP sera noté sous la forme d'une soutenance une semaine après la dernière séance de TP, ainsi que d'un rendu des codes sources du projet complet sous moodle.

Dans ce TP, vous aller reprendre le projet d'interaction avec la maquette vu précédemment en y ajoutant le système d'exploitation FreeRTOS.

Vous trouverez sur Youtube la série dédiée à FreeRTOS vous permettant d'intégrer le système d'exploitation dans votre projet.

Pour traiter ce TP, vous allez reprendre vos TP précédents en plus de créer un nouveau projet incluant FreeRTOS. C'est à vous de choisir comment organiser les tâches en fonction des contraintes des précédents TP.

2 Création des tâches, et communication inter-tâches

Validez l'utilisation de chaque éléments : Tâches, sémaphores, signaux, mutex... au fur et à mesure de leur création afin d'éviter de nombreux bugs qui pourraient être issus d'une compilation massive de différents codes.

1. En reprenant les TP précédents, proposez un découpage en tâches avec leur priorités des différentes fonctionnalités demandées.
2. Définissez les différentes Sémaphores, Mutexts, Signaux, Timers... A mettre en place pour gérer le changement de contexte entre chaque tâches.
3. Définissez les interruptions matériels nécessaires pour votre projet et le leur lien avec les tâches qui les requièrent.

3 Projet global sous FreeRTOS

1. Refaites le projet de la maquette avec votre binôme en communiquant avec les bus LIN et CAN sous FreeRTOS.
2. Optimisez votre programme pour que celui ci prennent le moins de place possible en mémoire, soit le moins bloquant possible sur certaines tâches et maximise le temps d'exécution de la tâche IDLE sans en pâtir sur les fonctionnalités demandées.

4 Optimisation énergétique de la carte électronique

1. En reprenant le cours et la documentation technique du STM32F407, utilisez les fonctions d'endormissement du microcontrôle dans FreeRTOS pour optimiser la gestion de l'énergie de celui ci.

2. Avec l'outil de diagnostic de CubeMX, faites une simulation de la consommation énergétique de la carte en fonction de votre programme et dimensionnez une source d'énergie pour pouvoir alimenter votre carte pendant une durée de 1 an grâce au cours. Vous justifierez vos calculs et résultats.